

Lecture 3: Introduction to GitHub and R Basics

Heather Mattie, PhD

September 9, 2025

Recipe of the Day!

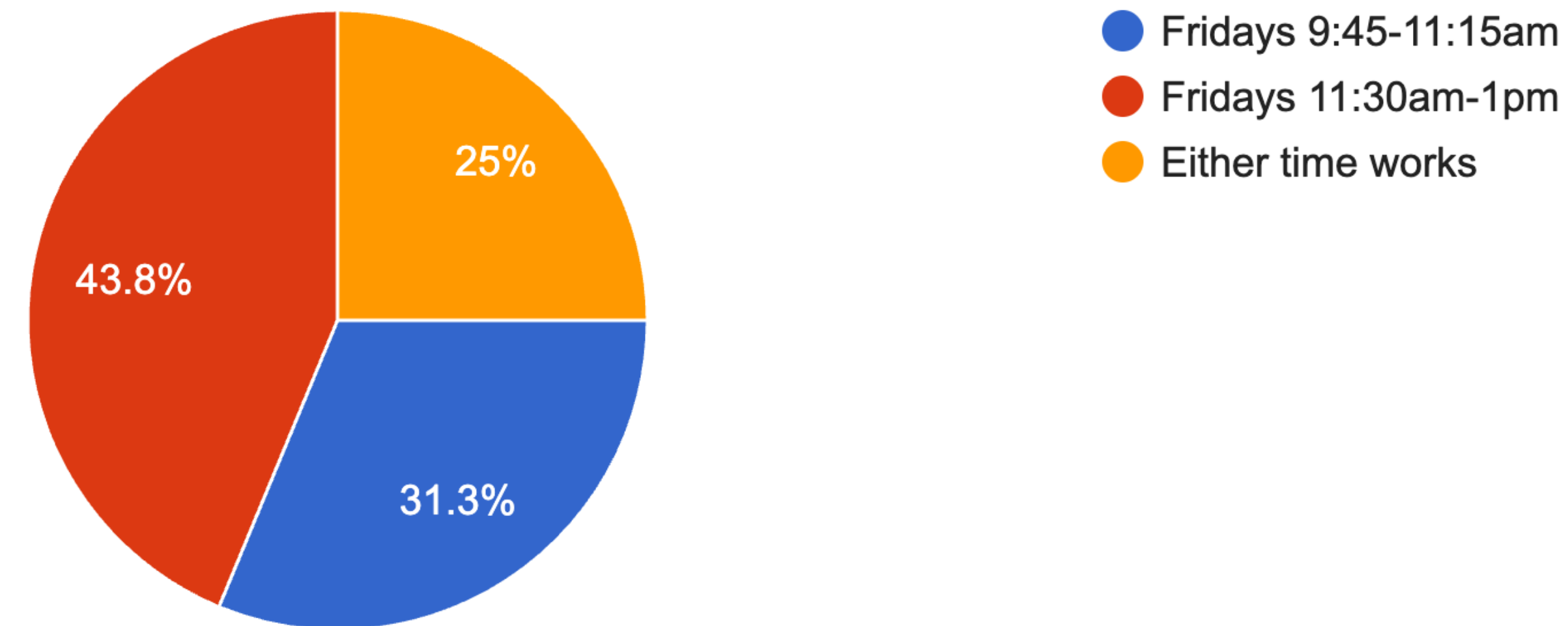
Lemon blueberry scones



Lab survey results

Which time would you prefer for the lab session? If either time would work for your schedule, please choose the "Either time works" option. Lab will be on Zoom and will be recorded.

16 responses



Office Hours and Lab

Office Hours

Day	Time	Staff	Location
Monday	10-11am	Sajia	FXB G10
Tuesday	1-2pm	Heather	Building 1, 4 th floor, Room 421A
Wednesday	2:30-3:30pm	Carmen	Kresge 201 except on 10/1 where it will be held in Kresge 502
Thursday	12-1pm	Claire	Kresge 205

Lab

Day	Time	Location
Friday	TBD	Zoom

***Lab will not be held every week**



Announcements

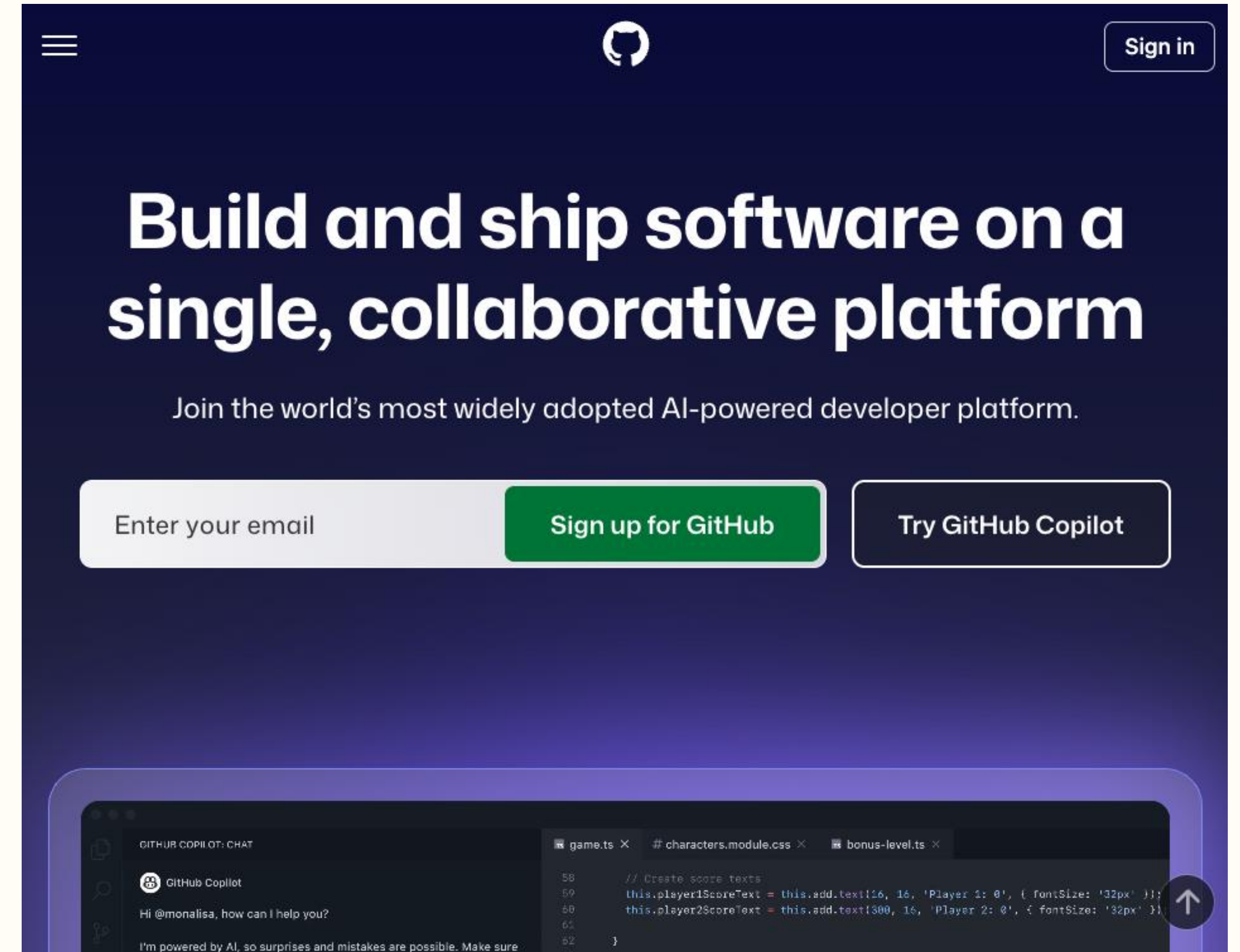
- Heather's office hour this week (tomorrow) will be 1:30-2pm only
 - Building 1, 4th floor, room 421A
- Lab will be held this week
- Homework #1 due Friday, September 19th by 11:59pm
 - May change depending on pacing
 - Can use up to 2 late days

Introduction to GitHub

GitHub

GitHub is a cloud-based platform for software developers and others to build, store, manage, and share code and projects using Git's version control system. It acts as a central hub for collaboration, allowing teams to

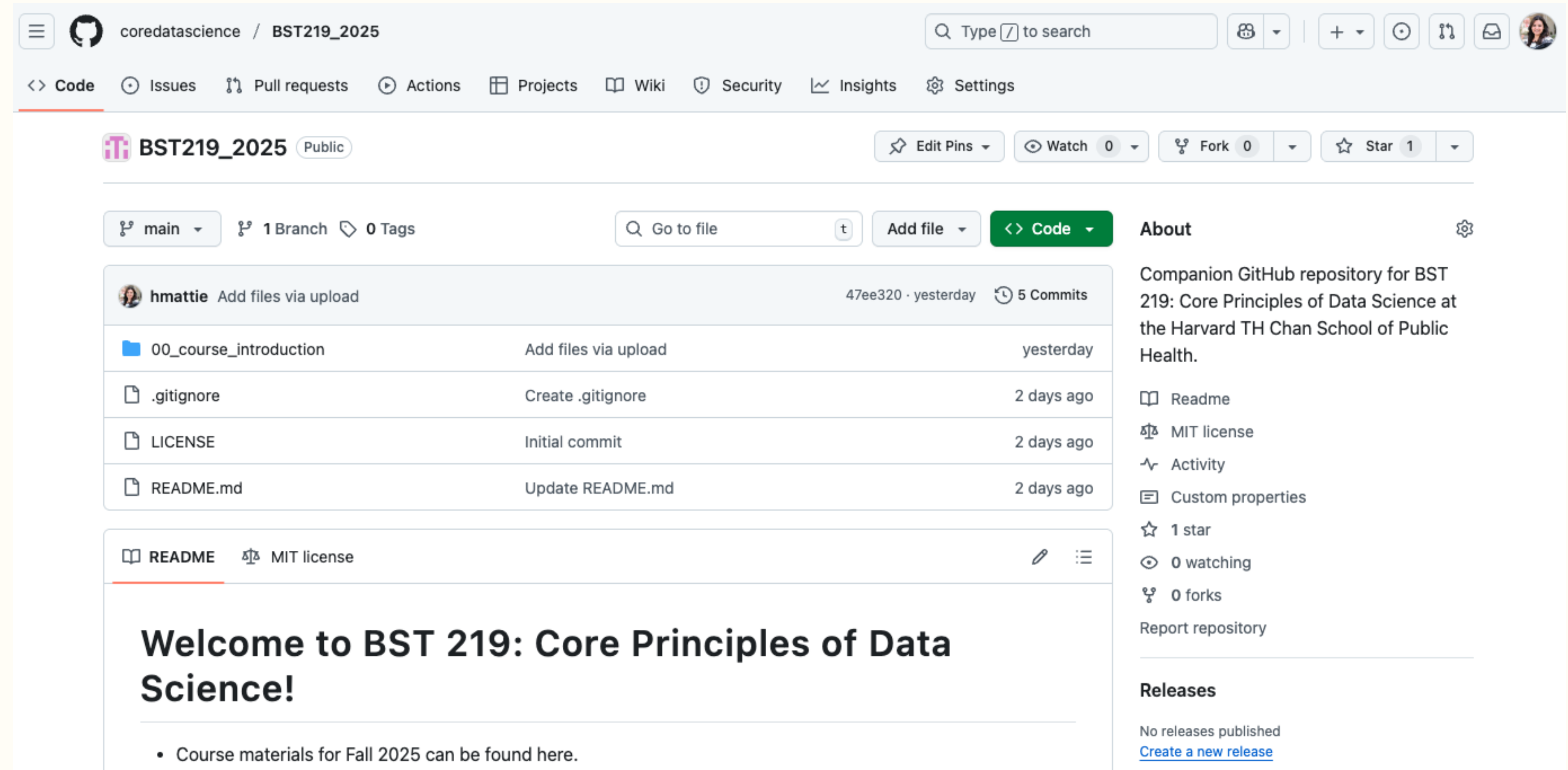
- Track changes
- Work on separate versions at the same time
- Suggest and review contributions
- Manage project tasks



GitHub

What will we use GitHub for?

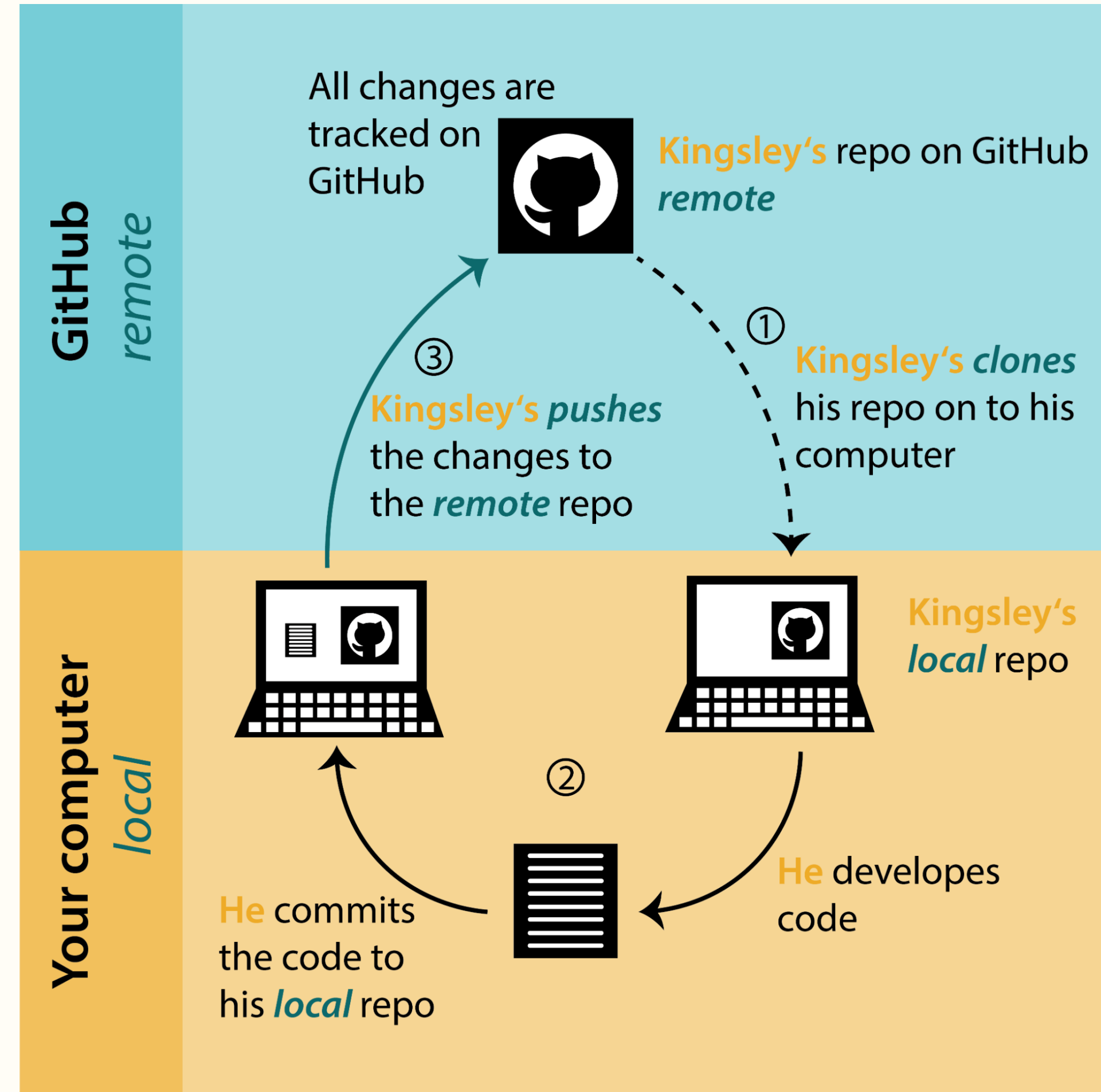
- All course materials
- All of your assignments



GitHub

The main reasons we will be using GitHub

- Version control so you don't lose any work
- Always good to be exposed to a tool commonly used in data science



GitHub

- GitHub has specific terminology it would be useful to remember.
- Different commands are used for different steps of the version control process. We will only be using a few of them in this course.

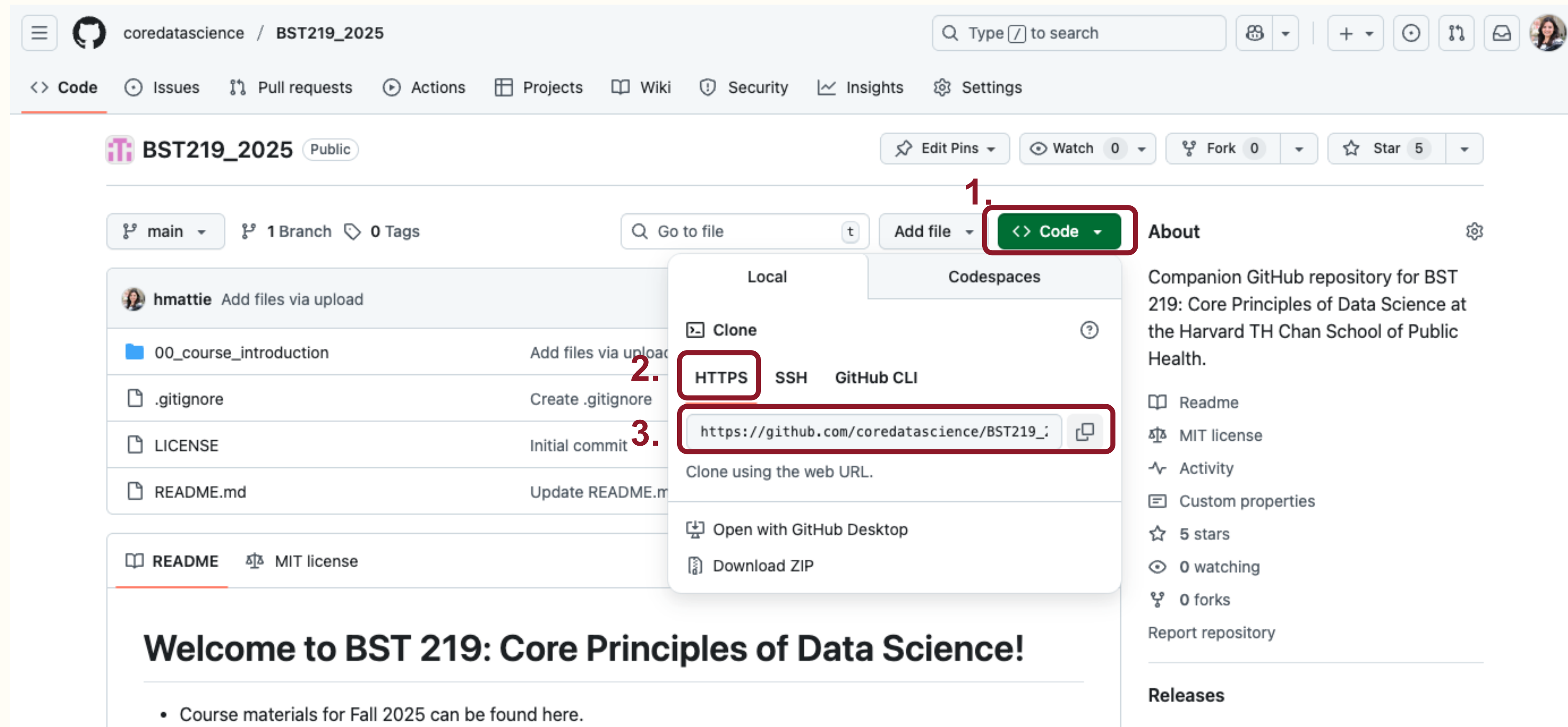
Definitions

- **repo** - short for repository
- **repository** - the directory or folder that is under version control
- **local** - the repository on your computer
- **remote** - the repository on GitHub
- **commit** - take a snapshot of one or more files in the repository
- **push** - send commits from the local repo to the remote repo
- **pull** - retrieve commits from the remote repo to the local repo
- **.gitignore** a file that tells git which files or types of files you don't want to commit

Accessing GitHub in RStudio

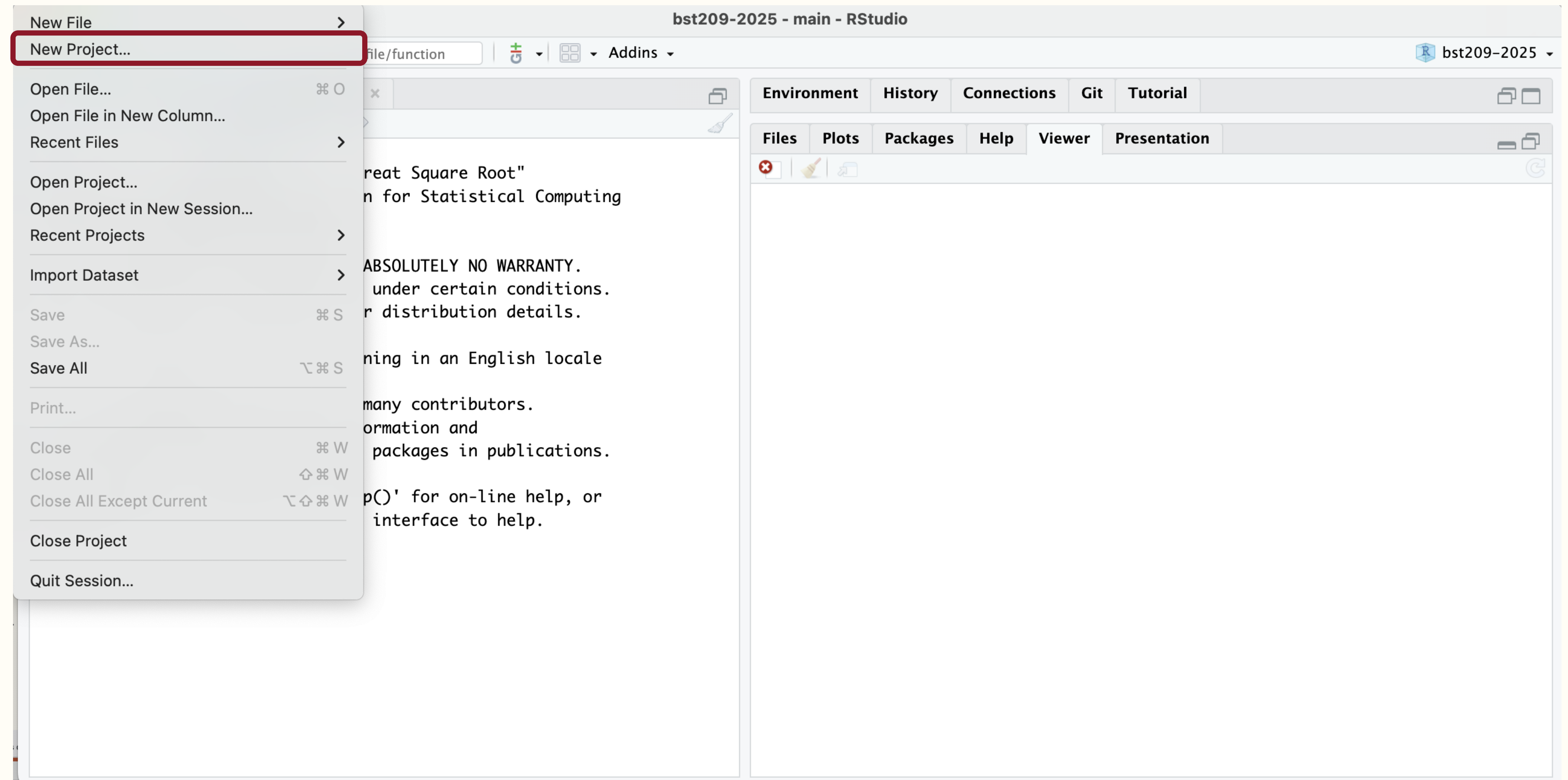
We won't be using the Terminal or Command Line to work with GitHub - we'll be working inside RStudio.

Step 1: Clone the repository



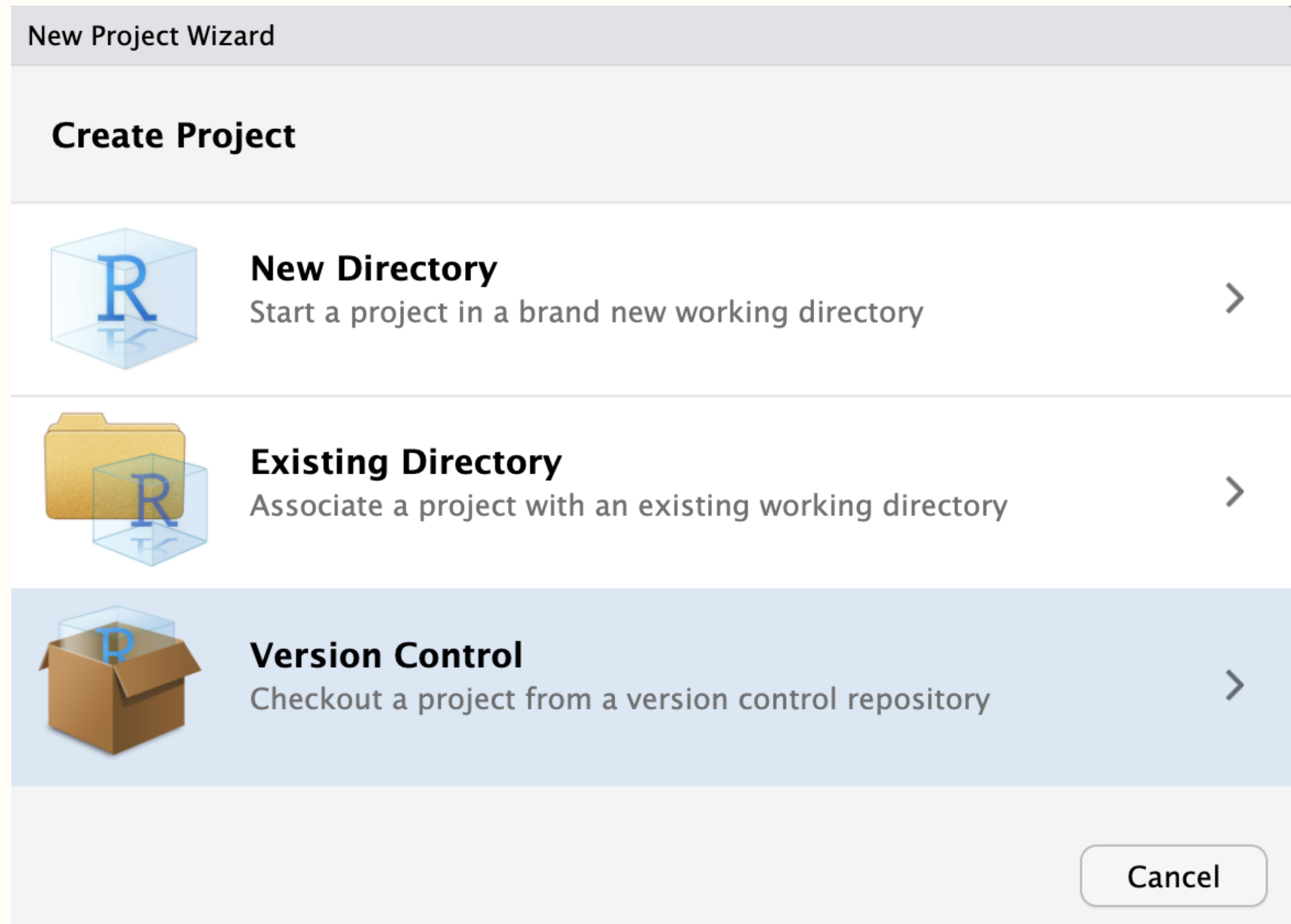
Accessing GitHub in RStudio

Step 2: Open RStudio and select New Project



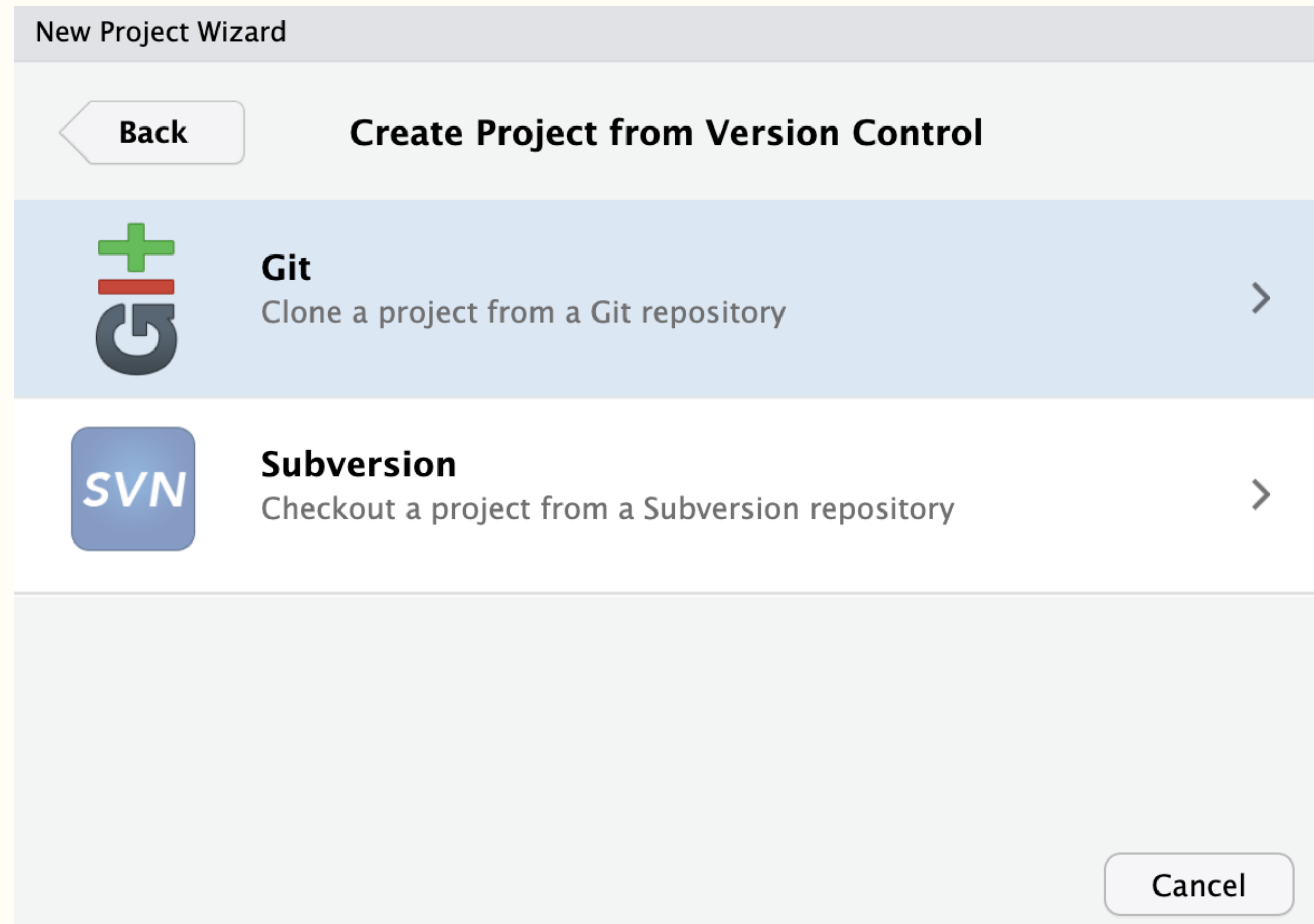
Accessing GitHub in RStudio

Step 3: Select Version Control



Accessing GitHub in RStudio

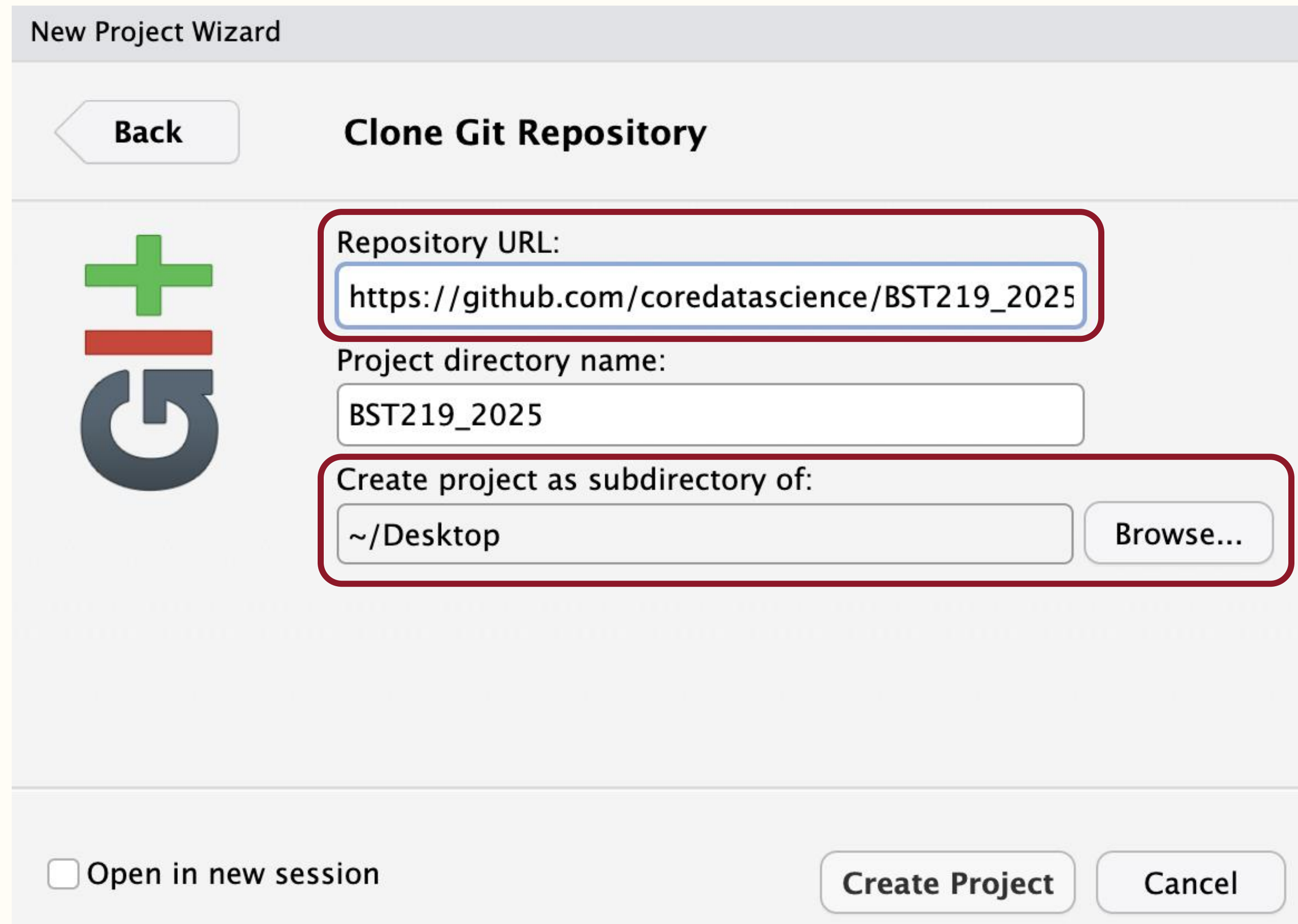
Step 4: Select Git



Accessing GitHub in RStudio

Step 5: Paste the repository link you copied on GitHub

This will become a folder on your computer - take note of where you are saving it!



The screenshot shows the 'New Project Wizard' dialog box in RStudio, specifically the 'Clone Git Repository' step. The dialog has a title bar 'New Project Wizard' and a 'Back' button. On the left is a large green plus sign over a grey 'G' logo. The main area contains three input fields: 'Repository URL:' with the value 'https://github.com/coredatascience/BST219_2025', 'Project directory name:' with the value 'BST219_2025', and 'Create project as subdirectory of:' with the value '~/Desktop'. A 'Browse...' button is next to the last field. At the bottom, there is a checkbox 'Open in new session' and two buttons: 'Create Project' and 'Cancel'.

New Project Wizard

Back Clone Git Repository

Repository URL:
https://github.com/coredatascience/BST219_2025

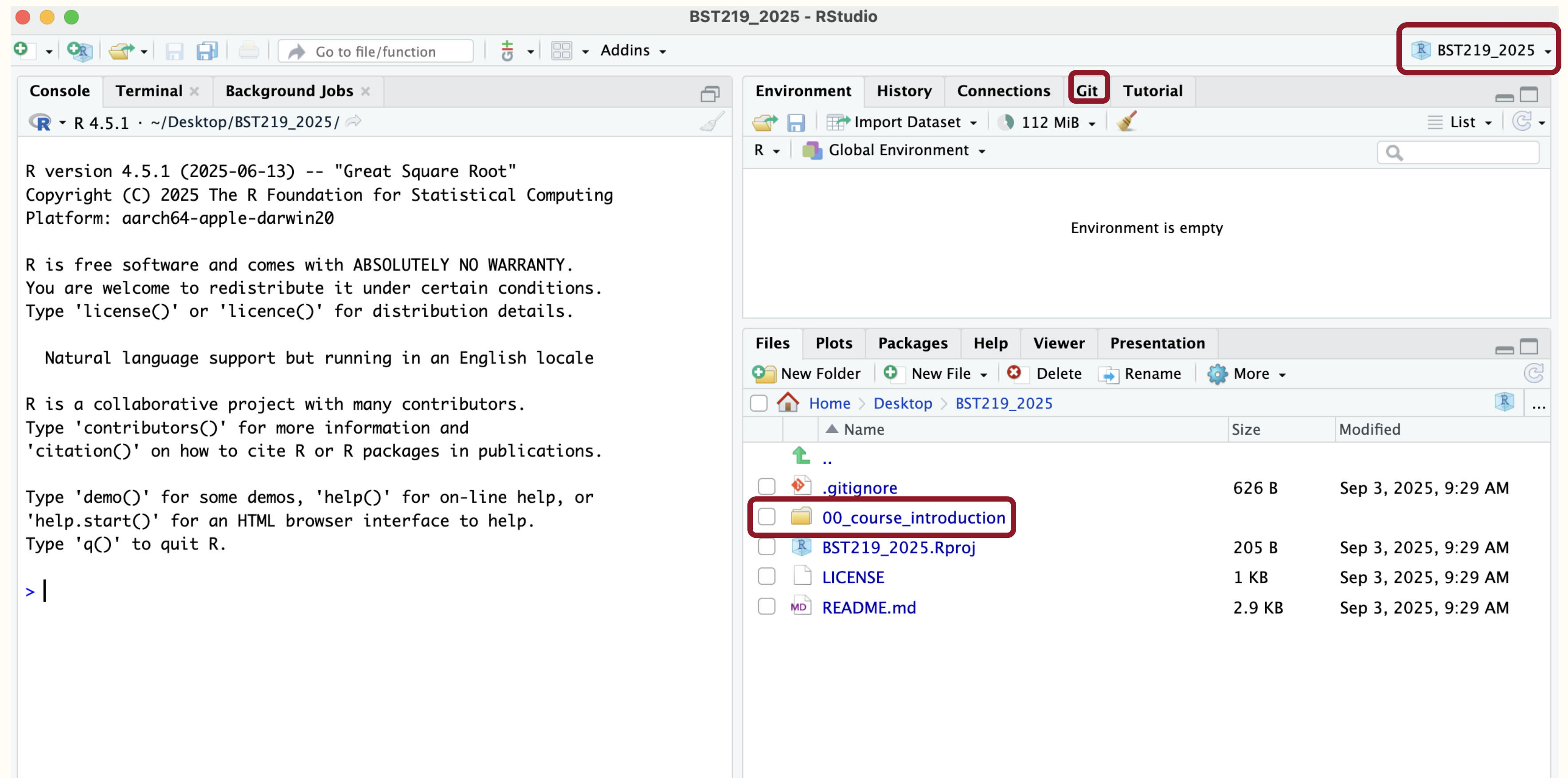
Project directory name:
BST219_2025

Create project as subdirectory of:
~/Desktop Browse...

☐ Open in new session Create Project Cancel

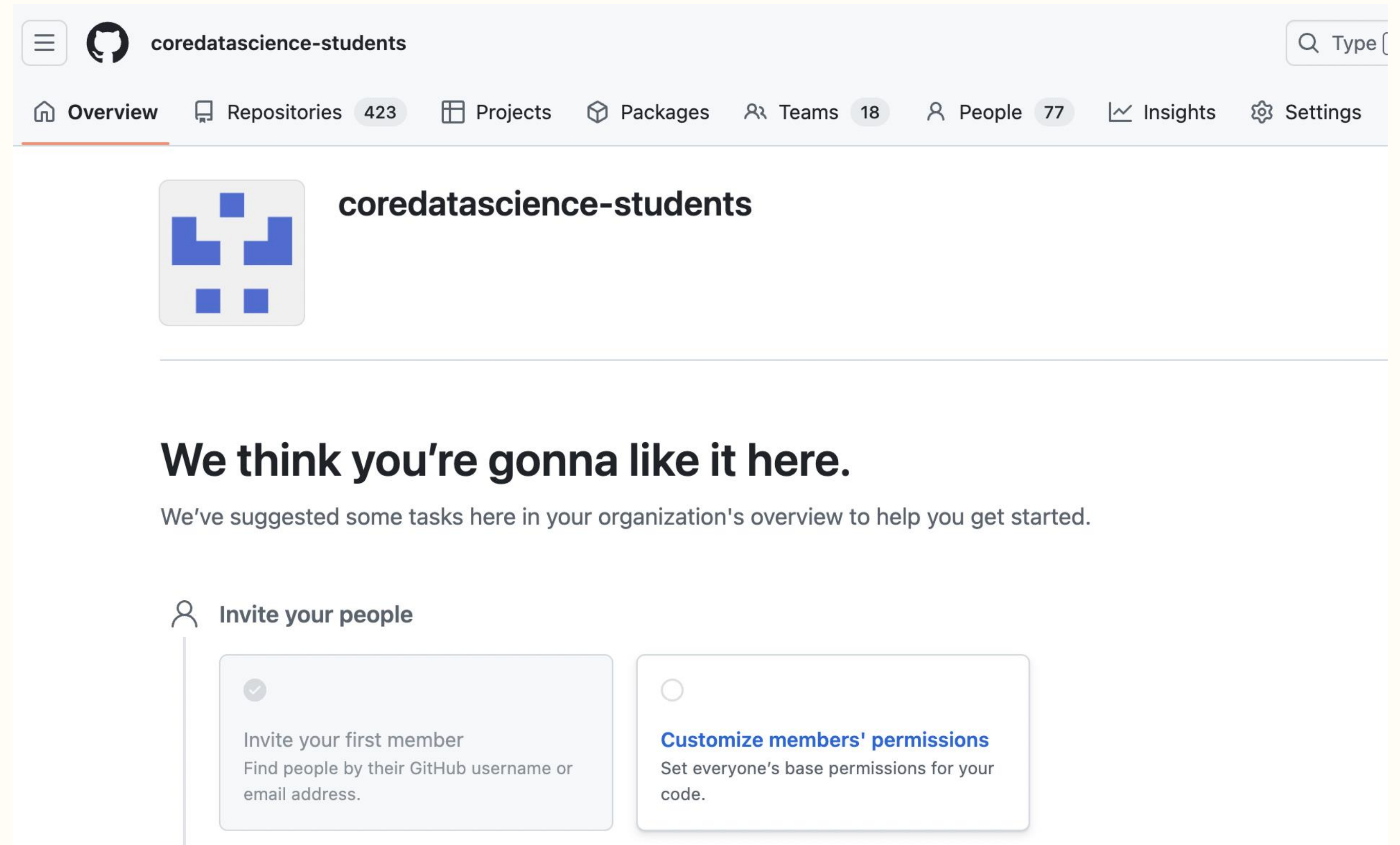
Accessing GitHub in RStudio

The project should open and you should see the project (repo) name and all folders and files in the repo.



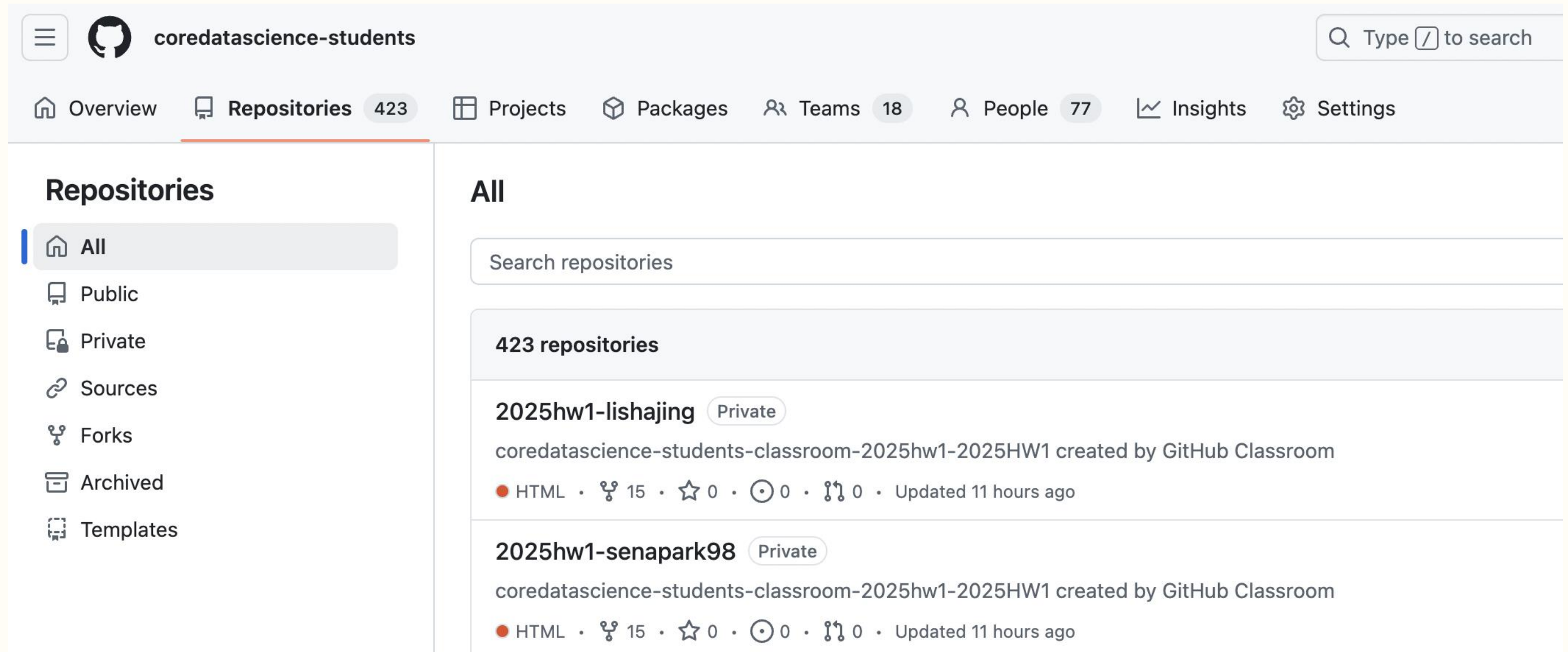
Working on and submitting homework

The steps to clone your homework repositories are exactly the same as the course materials repository, except you will go to the [student repositories page](#).



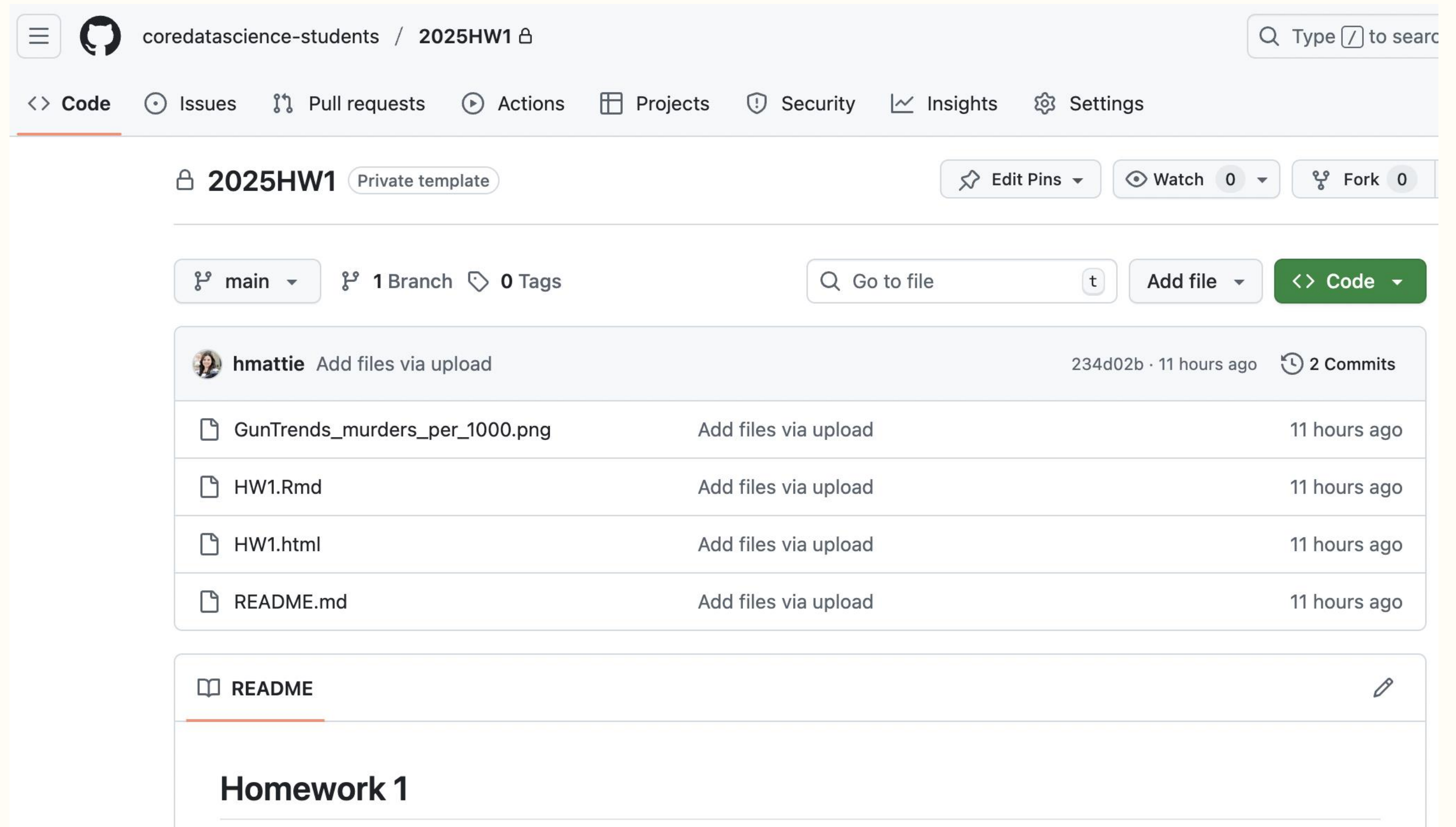
Working on and submitting homework

The steps to clone your homework repositories are exactly the same as the course materials repository, except you will go to the student repositories page.



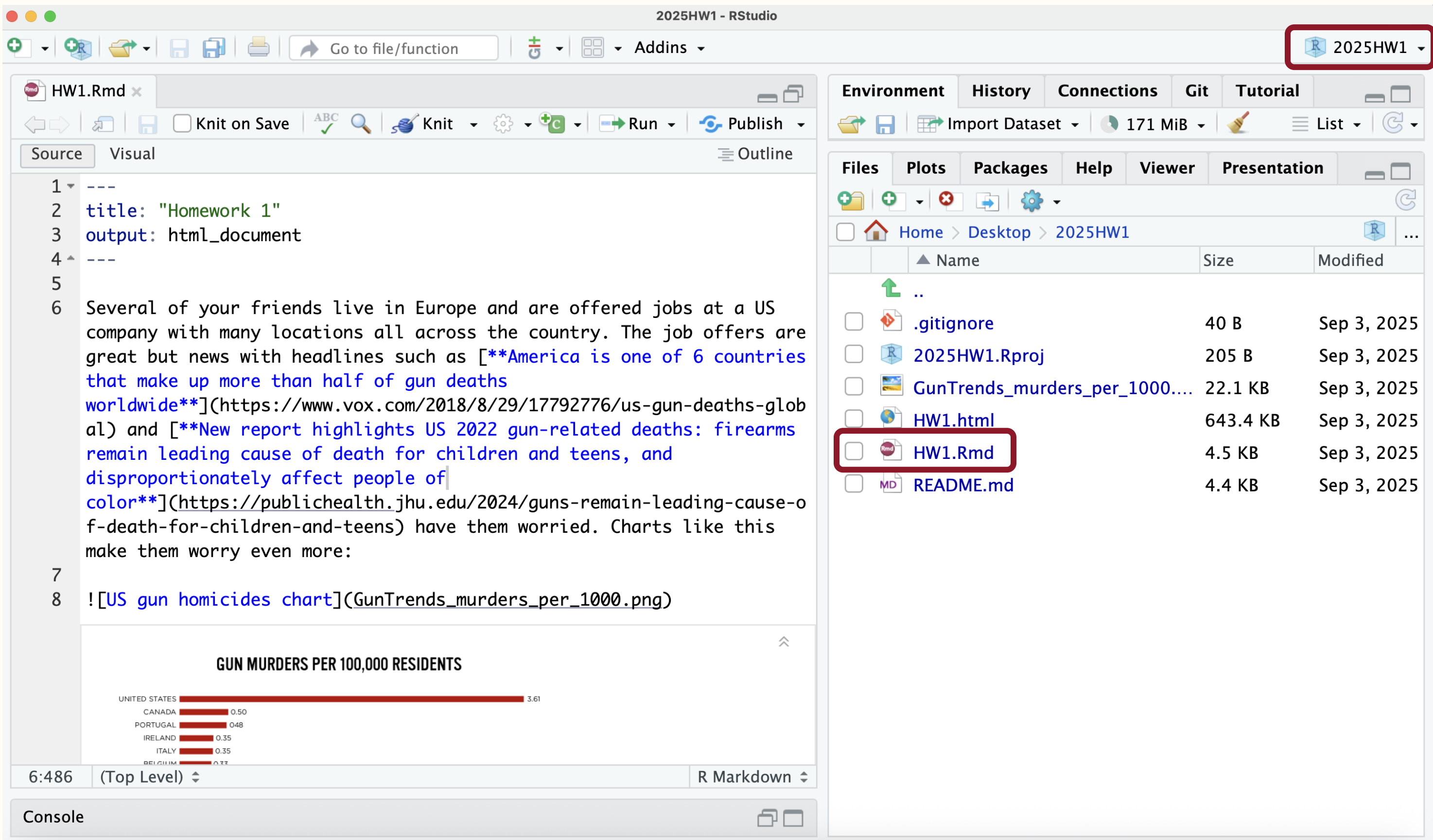
Working on and submitting homework

The steps to clone your homework repositories are exactly the same as the course materials repository, except you will go to the [student repositories page](#).



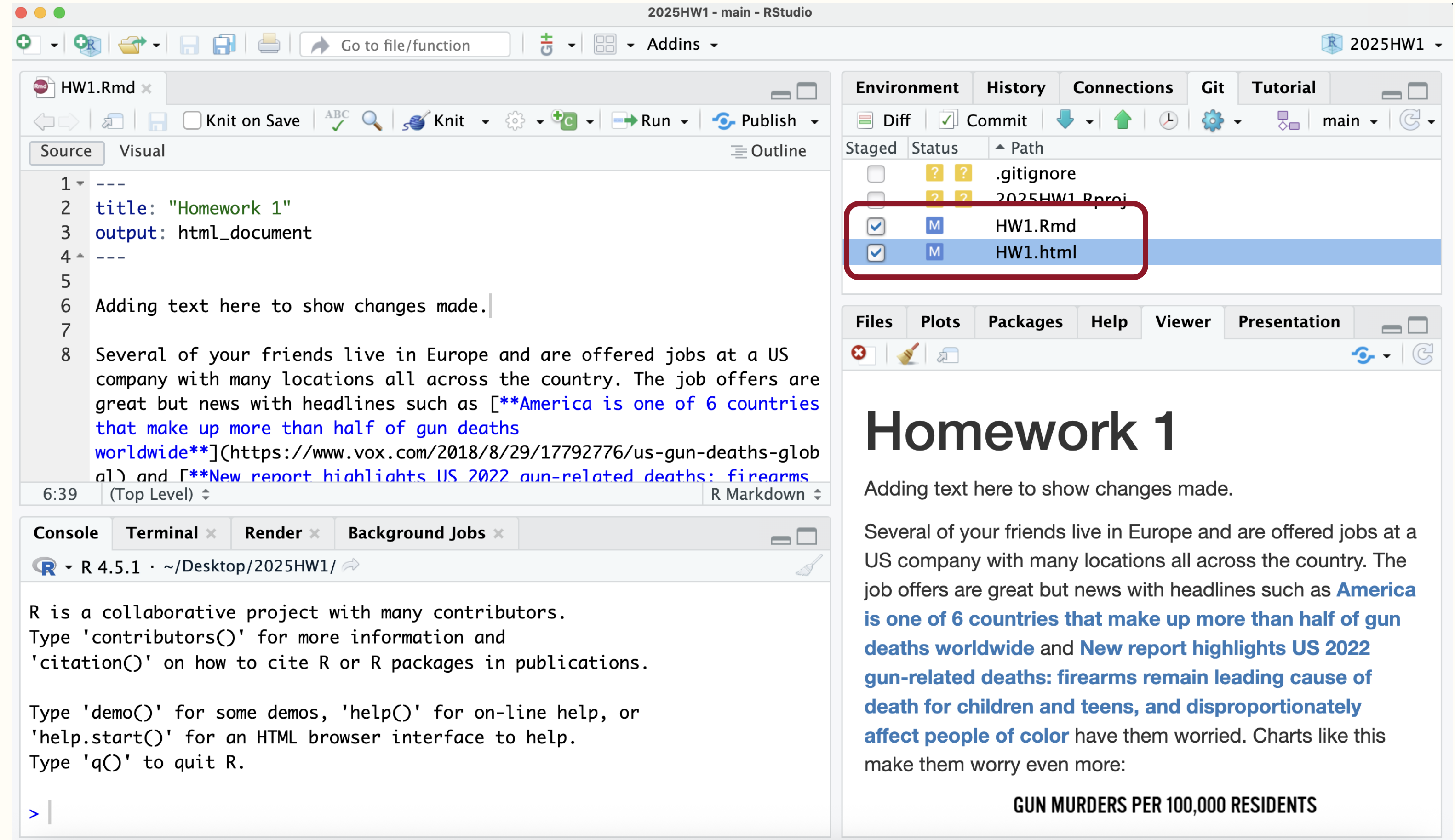
Working on and submitting homework

Once you have cloned your homework repository, click on the .Rmd file to work on your assignment.



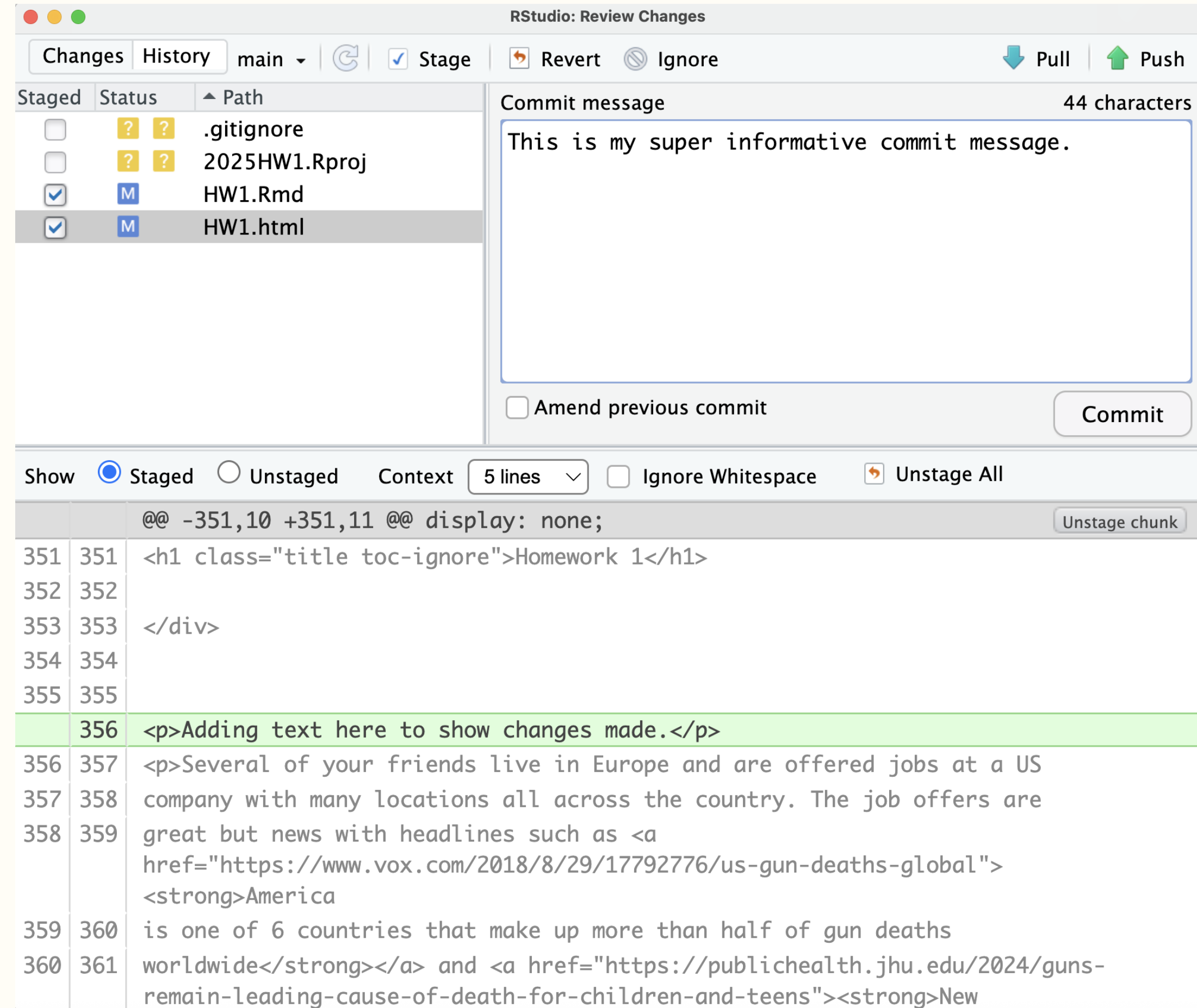
Working on and submitting homework

When you are done working on the assignment and want to save your progress, or submit the final version, knit the .Rmd file and then click on the **Git** tab. Then click the boxes next to the .Rmd and .html file names, and press the **Commit** button.



Working on and submitting homework

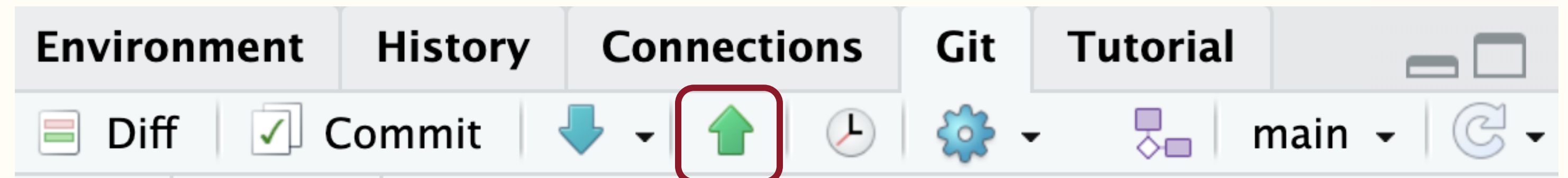
A new window will pop up. Write a commit message, e.g.,
“Question 1 answered”
or “Final version” and
press **Commit**.



Working on and submitting homework

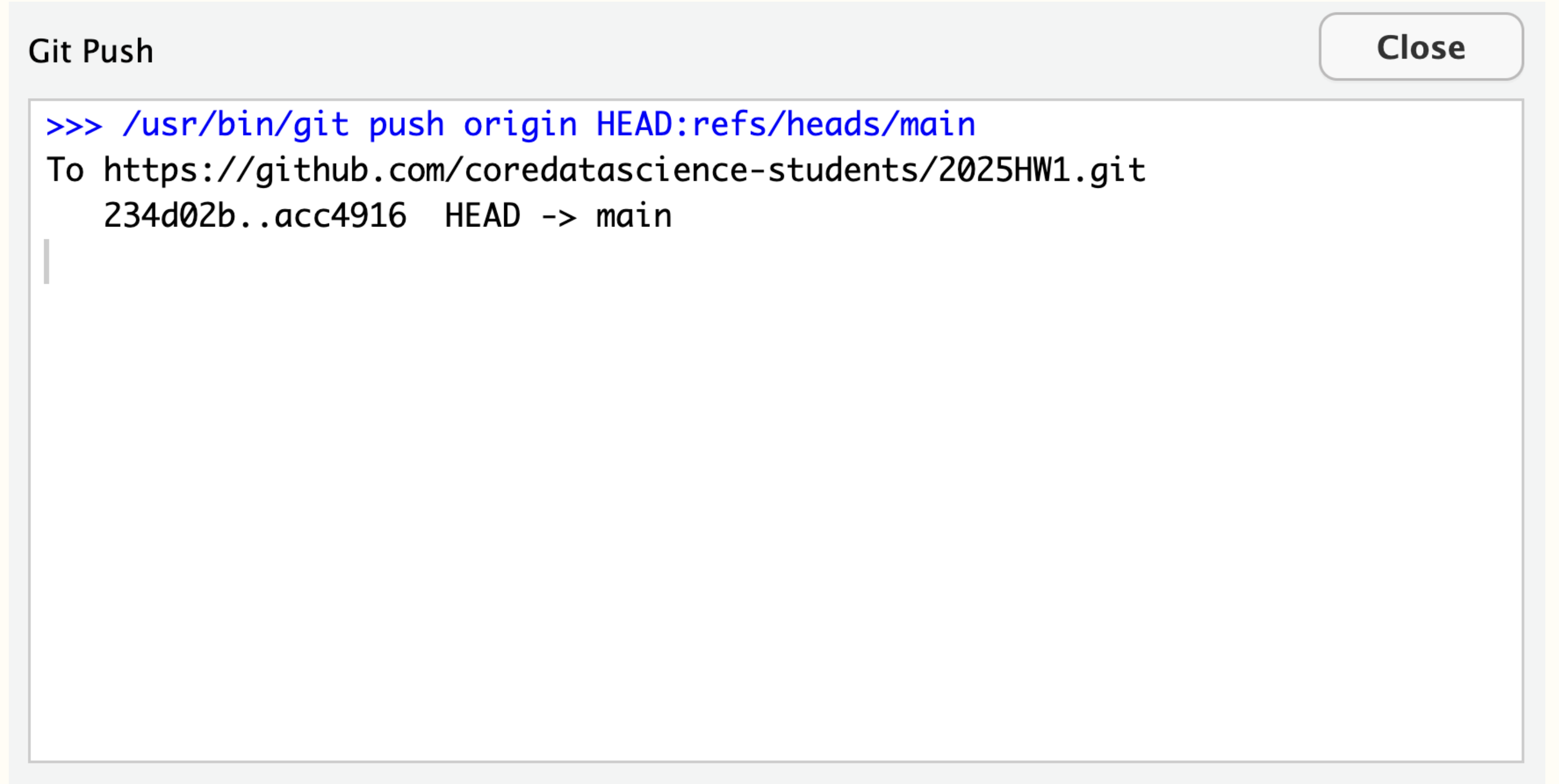
A new window will pop up – but you aren't done! Close this window and then click the **Push** green button.

```
Git Commit Close  
  
>>> /usr/bin/git commit -F /var/folders/ml/nsp77dv93d1f9x73bxzvzh100000gp/T/Rt  
[main acc4916] This is my super informative commit message.  
2 files changed, 3 insertions(+)
```



Working on and submitting homework

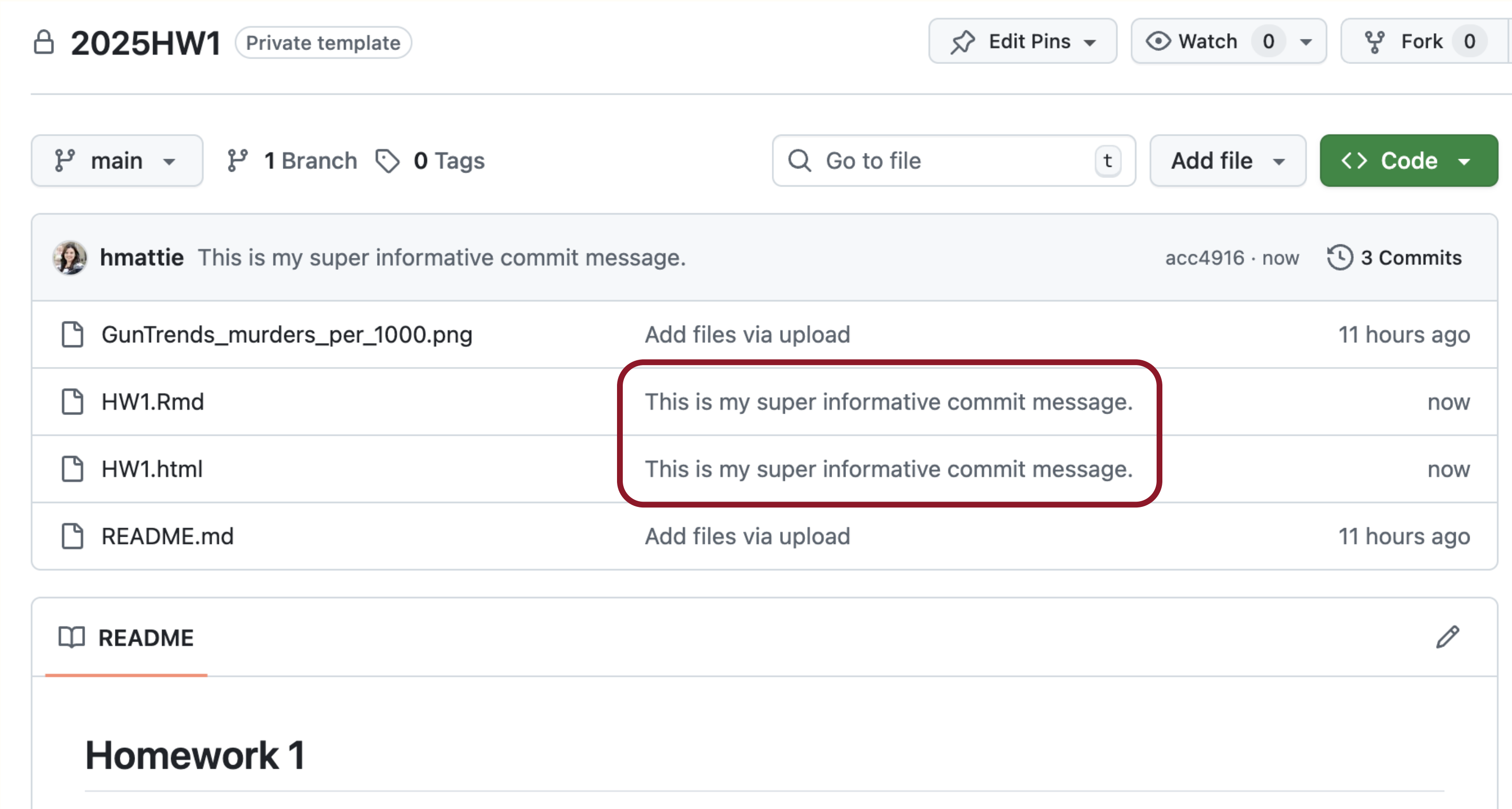
A new window will pop up that looks something like this. Now you are done!

A screenshot of a terminal window titled "Git Push" with a "Close" button in the top right corner. The terminal shows a successful git push command. The text in the terminal is: >>> /usr/bin/git push origin HEAD:refs/heads/main, To https://github.com/coredatascience-students/2025HW1.git, 234d02b..acc4916 HEAD -> main.

```
Git Push
Close
>>> /usr/bin/git push origin HEAD:refs/heads/main
To https://github.com/coredatascience-students/2025HW1.git
234d02b..acc4916 HEAD -> main
```

Working on and submitting homework

You can check that you pushed successfully by going back to your repository on GitHub, refreshing the page, and looking to see your Commit message shows up.



Working on and submitting homework

We encourage you to knit, commit, and push often to save your progress!

2025HW1Private template

Edit PinsWatch 0Fork 0

main1 Branch0 Tags

Go to file

Add file

Code

hmttie

This is my super informative commit message.

acc4916 · now

3 Commits

GunTrends_murders_per_1000.png	Add files via upload	11 hours ago
HW1.Rmd	This is my super informative commit message.	now
HW1.html	This is my super informative commit message.	now
README.md	Add files via upload	11 hours ago

README

Homework 1



Homework #1 details

- Your plots can be created using base R. You do not need to use `ggplot2` - yet. You will for all other assignments.
- You do not need to use the pipe (`%>%`) for this homework. You will for all other assignments.
- Comment your code, i.e., provide short explanations of what each chunk of code is doing. This is good coding practice. Everyone who reads your code – including your future self – will thank you.

General homework tips

- **Do not** write text answers inside of a code chunk. Code chunks are for code (and code comments) only. This is the main reason we are using R Markdown files!
- Be sure your homework knits successfully and that you commit and push **BOTH** the .Rmd and .html files.
- **Do not** print out an entire dataset. Use the `head()` or `glimpse()` functions.

R Basics

R Basics Roadmap

1. Data types
2. Vectors
3. Sorting
4. Indexing
5. Basic plots
6. Programming basics

R Basics

Data Types

Heather Mattie, PhD

Data types

A **data type** is a classification that specifies the kind of value a variable can hold and how R should interpret and handle it

- Numeric (integer or decimal)
- Text (string, character)
- True/False value (Boolean, logical)
- Data frame
- List

Data types

Integer

```
x <- 2  
class(x)
```

```
## [1] "numeric"
```

Decimal

```
y <- 3.14  
class(y)
```

```
## [1] "numeric"
```

Text

```
z <- "Boston"  
class(z)
```

```
## [1] "character"
```

Logical

```
a <- TRUE  
class(a)
```

```
## [1] "logical"
```

Data frame

```
library(dslabs)  
data(gapminder)  
d <- gapminder  
class(d)
```

```
## [1] "data.frame"
```

List

```
record <- list(name = "John Doe",  
               student_id = 1234,  
               grades = c(95, 82, 91, 97, 93),  
               final_grade = "A")  
class(record)
```

```
## [1] "list"
```


Examining objects

The function **str** displays more about the structure of an object

```
library(dslabs)
data(gapminder)
str(gapminder)
```

```
## 'data.frame':  10545 obs. of  9 variables:
## $ country      : Factor w/ 185 levels "Albania","Algeria",...: 1 2 3 4 5 6 7 8 9 10 ...
## $ year         : int  1960 1960 1960 1960 1960 1960 1960 1960 1960 1960 ...
## $ infant_mortality: num  115.4 148.2 208 NA 59.9 ...
## $ life_expectancy : num  62.9 47.5 36 63 65.4 ...
## $ fertility     : num  6.19 7.65 7.32 4.43 3.11 4.55 4.82 3.45 2.7 5.57 ...
## $ population    : num  1636054 11124892 5270844 54681 20619075 ...
## $ gdp           : num  NA 1.38e+10 NA NA 1.08e+11 ...
## $ continent     : Factor w/ 5 levels "Africa","Americas",...: 4 1 1 2 2 3 2 5 4 3 ...
## $ region        : Factor w/ 22 levels "Australia and New Zealand",...: 19 11 10 2 15 21 2 1 22 21 ...
```

Examining objects

The function **head** displays the first 6 rows and all columns of a data frame

```
head(gapminder)
```

```
##           country year infant_mortality life_expectancy fertility
## 1      Albania 1960      115.40           62.87           6.19
## 2      Algeria 1960      148.20           47.50           7.65
## 3       Angola 1960      208.00           35.98           7.32
## 4 Antigua and Barbuda 1960      NA           62.97           4.43
## 5      Argentina 1960      59.87           65.39           3.11
## 6      Armenia 1960      NA           66.86           4.55
## population      gdp continent      region
## 1    1636054      NA    Europe Southern Europe
## 2    11124892 13828152297    Africa Northern Africa
## 3     5270844      NA    Africa Middle Africa
## 4      54681      NA Americas Caribbean
## 5    20619075 108322326649 Americas South America
## 6    1867396      NA     Asia Western Asia
```

The accessor

The **accessor operator \$** is used to access different variables.

Most commonly, it is used to access the values in a column in a data frame.

```
library(dslabs)
data(admissions)
names(admissions)
```

```
## [1] "major"      "gender"     "admitted"   "applicants"
```

```
admissions$major
```

```
## [1] "A" "B" "C" "D" "E" "F" "A" "B" "C" "D" "E" "F"
```


Vectors

The `admissions$major` object is not one character, but several – it is a **vector**.

- The number of entries in a vector can be displayed with the **length** function

```
majors <- admissions$major  
length(majors)
```

```
## [1] 12
```

```
class(majors)
```

```
## [1] "character"
```


Factors

Factors are a data type useful for storing categorical data.

R stores these *levels* (categories) as integers but keeps track of the labels by keeping a map of the integer to the level. This is more memory efficient than storing all the characters.

```
head(gapminder)
```

```
##           country year infant_mortality life_expectancy fertility
## 1      Albania 1960      115.40           62.87           6.19
## 2      Algeria 1960      148.20           47.50           7.65
## 3       Angola 1960      208.00           35.98           7.32
## 4 Antigua and Barbuda 1960           NA           62.97           4.43
## 5      Argentina 1960       59.87           65.39           3.11
## 6      Armenia 1960           NA           66.86           4.55
## population      gdp continent      region
## 1    1636054         NA    Europe Southern Europe
## 2    11124892 13828152297    Africa Northern Africa
## 3     5270844         NA    Africa Middle Africa
## 4      54681         NA Americas Caribbean
## 5    20619075 108322326649 Americas South America
## 6    1867396         NA     Asia Western Asia
```

```
class(gapminder$country)
```

```
## [1] "factor"
```

Summary

- A data type specifies the kind of value a variable can be and how the R should interpret and handle it
- The **str** function displays the structure of an object
- The **head** function displays the first 6 rows of a data frame
- The accessor **\$** is used to access different variables

R Basics

Vectors

Heather Mattie, PhD

Creating vectors

Vectors are created using the concatenate function **c**.

Vectors with text or characters as entries must have quotation marks around each entry.

- Without quotation marks, R will look for variables with those names
- An error will be produced

```
codes <- c(380, 124, 818)
codes
```

```
## [1] 380 124 818
```

```
country <- c("italy", "canada", "egypt")
country
```

```
## [1] "italy" "canada" "egypt"
```

```
country <- c(italy, canada, egypt)
```


Names

It might be useful to **name** the entries of a vector

- Connect the entry with a name for the entry with an equal sign
- Names can be coded with or without quotation marks
- Names can also be assigned using the **names** function

```
codes <- c(italy = 380, canada = 124, egypt = 818)
codes
```

```
## italy canada egypt
##    380    124    818
```

```
names(codes)
```

```
## [1] "italy" "canada" "egypt"
```

```
codes <- c("italy" = 380, "canada" = 124, "egypt" = 818)
codes
```

```
## italy canada egypt
##    380    124    818
```

```
codes <- c(380, 124, 818)
country <- c("italy", "canada", "egypt")
names(codes) <- country
codes
```

```
## italy canada egypt
##    380    124    818
```

Sequences

The **seq** function creates vector sequences

- The first argument defines the start
- The second argument defines the end
- The default increment is to increase by 1 but this can be customized
- There is shorthand for consecutive numbers
- The default class of the entries in a sequence is integer but will be numeric if decimal numbers are part of the sequence

```
seq(1, 10)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(1, 10, 2)
```

```
## [1] 1 3 5 7 9
```

```
1:10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
class(1:10)
```

```
## [1] "integer"
```

```
class(seq(1, 10, 0.5))
```

```
## [1] "numeric"
```

Subsetting

Square brackets are used to access specific elements in a vector

- The position of the entry in the vector, or the name of the entry can be used to access it
- A vector or sequence of numbers can also be used inside the square brackets

```
codes
```

```
## italy canada egypt  
## 380 124 818
```

```
codes[2]
```

```
## canada  
## 124
```

```
codes[c(1,3)]
```

```
## italy egypt  
## 380 818
```

```
codes[1:2]
```

```
## italy canada  
## 380 124
```

```
codes["canada"]
```

```
## canada  
## 124
```

```
codes[c("egypt", "italy")]
```

```
## egypt italy  
## 818 380
```


Coercion

Coercion is R attempting to be flexible with data types

- The entries in a vector must be the same data type
- If an entry isn't the type expected, R will guess what it is and **coerce** the value to be a certain type if it can
- Otherwise, it will give an error
- Numeric entries **can** be coerced into characters
- Character entries **cannot** be coerced into numbers

```
x <- c(1, "canada", 3)
x
```

```
## [1] "1"      "canada" "3"
```

```
class(x)
```

```
## [1] "character"
```

```
x <- 1:5
y <- as.character(x)
y
```

```
## [1] "1" "2" "3" "4" "5"
```

```
as.numeric(y)
```

```
## [1] 1 2 3 4 5
```


Not Availables (NA)

When a coercion function encounters an impossible case, it gives a warning and turns the entry into an **NA**

- NA stands for not available
- NA is also used to denote missing data

```
x <- c("1", "b", "3")  
as.numeric(x)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] 1 NA 3
```

Vector arithmetic

Vector arithmetic involves performing mathematical operations on vectors

- Addition, subtraction, multiplication, division, etc.
- In R, arithmetic operations on vectors occur *element wise*

```
heights <- c(69, 62, 66, 70, 70, 73, 67, 73, 67, 70)
```

```
heights * 2.54
```

```
## [1] 175.26 157.48 167.64 177.80 177.80 185.42 170.18 185.42 170.18 177.80
```

```
heights - 69
```

```
## [1] 0 -7 -3 1 1 4 -2 4 -2 1
```

Two vectors

If we have two vectors of the same length, and we sum them, they are added entry by entry

- The same holds for subtraction, multiplication, and division

$$\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} + \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} = \begin{pmatrix} a + e \\ b + f \\ c + g \\ d + h \end{pmatrix}$$

Summary

- Vectors are objects with multiple entries that can be created with the concatenate function
- All entries in a vector must be of the same data type
- The entries in a vector can be named
- The **seq** function produces a sequence vector
- Square brackets are used to subset a vector
- R will use coercion if the data types of entries in a vector conflict

R Basics

Sorting

Heather Mattie, PhD

sort

The **sort** function sorts a vector in **increasing** order

- It does not indicate the placement, or index, of the values once they have been sorted

```
library(dslabs)
data(murders)
sort(murders$total)
```

```
## [1] 2 4 5 5 7 8 11 12 12 16 19 21 22 27 32
## [16] 36 38 53 63 65 67 84 93 93 97 97 99 111 116 118
## [31] 120 135 142 207 219 232 246 250 286 293 310 321 351 364 376
## [46] 413 457 517 669 805 1257
```

order

The **order** function takes a vector and returns a vector of indices

- The indices show how to sort the original vector
- It does not sort the data itself

```
x <- c(31, 4, 15, 92, 65)
sort(x)
```

```
## [1] 4 15 31 65 92
```

```
index <- order(x)
x[index]
```

```
## [1] 4 15 31 65 92
```

```
x
```

```
## [1] 31 4 15 92 65
```

```
order(x)
```

```
## [1] 2 3 1 5 4
```

order

```
library(dslabs)
data(murders)
ind <- order(murders$total)
murders$abb[ind]
```

```
## [1] "VT" "ND" "NH" "WY" "HI" "SD" "ME" "ID" "MT" "RI" "AK" "IA" "UT" "WV" "NE"
## [16] "OR" "DE" "MN" "KS" "CO" "NM" "NV" "AR" "WA" "CT" "WI" "DC" "OK" "KY" "MA"
## [31] "MS" "AL" "IN" "SC" "TN" "AZ" "NJ" "VA" "NC" "MD" "OH" "MO" "LA" "IL" "GA"
## [46] "MI" "PA" "NY" "FL" "TX" "CA"
```


Rate example using **order**

Vector arithmetic can be especially useful for creating new variables quickly

- Example: creating and comparing murder rates for each state rather than the total number of murders in a particular state

```
head(murders)
```

```
##      state abb region population total
## 1  Alabama AL  South   4779736    135
## 2   Alaska AK   West    710231     19
## 3  Arizona AZ   West   6392017    232
## 4  Arkansas AR  South   2915918     93
## 5 California CA   West  37253956   1257
## 6   Colorado CO   West   5029196     65
```

```
murder_rate <- murders$total / murders$population * 100000
```

```
murders$state[order(murder_rate)]
```

```
## [1] "Vermont"           "New Hampshire"    "Hawaii"
## [4] "North Dakota"      "Iowa"             "Idaho"
## [7] "Utah"              "Maine"            "Wyoming"
## [10] "Oregon"            "South Dakota"     "Minnesota"
## [13] "Montana"           "Colorado"         "Washington"
## [16] "West Virginia"     "Rhode Island"     "Wisconsin"
## [19] "Nebraska"          "Massachusetts"    "Indiana"
## [22] "Kansas"            "New York"         "Kentucky"
## [25] "Alaska"            "Ohio"             "Connecticut"
## [28] "New Jersey"        "Alabama"          "Illinois"
## [31] "Oklahoma"          "North Carolina"   "Nevada"
## [34] "Virginia"          "Arkansas"         "Texas"
## [37] "New Mexico"        "California"       "Florida"
## [40] "Tennessee"         "Pennsylvania"     "Arizona"
## [43] "Georgia"           "Mississippi"      "Michigan"
## [46] "Delaware"          "South Carolina"   "Maryland"
## [49] "Missouri"          "Louisiana"        "District of Columbia"
```

rank

For any given vector, the **rank** function gives you a vector with the rank of the first entry, second entry, etc. of the vector

```
x <- c(31, 4, 15, 92, 65)  
rank(x)
```

```
## [1] 3 1 2 5 4
```


Comparing sorting functions

original	sort	order	rank
31	4	2	3
4	15	3	1
15	31	1	2
92	65	5	5
65	92	4	4



Summary

- The **sort** function sorts a vector in increasing order
- The **order** function returns a vector of indices that sort the original vector
- The **rank** function returns a vector with the rank of each entry when the original vector is sorted